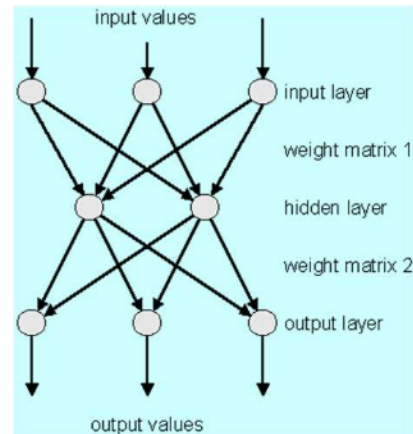


Multilayer Perceptron (1962)

- Perceptrons were developed in the 1950s and 1960s by the scientist **Frank Rosenblatt**, inspired by earlier work by Warren McCulloch and Walter Pitts.
- The Multi-Layer-Perceptron was introduced by **M. Minsky and S. Papert in 1969**.



Multilayer Perceptron can solve XOR:

- At the time Perceptron was published (1962), no one knew **how Multilayer Perceptrons could be made to learn**.
- Despite the identification of this important research problem, actual research in Perceptron came to halt during 1970s, it was called period of frustration in ANN research.
- This field saw little interest until 1980s, when several learning procedures for Multilayer Perceptron were perceived.

1969 – 1982

(Era of frustration in ANN research)

- Following an initial period of enthusiasm (1943-1958), the field survived a period of frustration and disrepute.
- During this period when funding and professional support was minimal, important advances were made by relatively few researchers.
- Minsky and Papert, published a book (in 1969) in which they summed up a general feeling of frustration (against neural networks) among researchers, and was thus accepted by most without further analysis.
- **Lack of suitable learning law for a MLP network had put breaks on the development of NN models for pattern recognition tasks for nearly 15 years till 1984.**

Backpropagation Network

The Backpropagation Net was first introduced by G.E. Hinton, E. Rumelhart and R.J. Williams in 1986

Type:

Feedforward

Neuron layers:

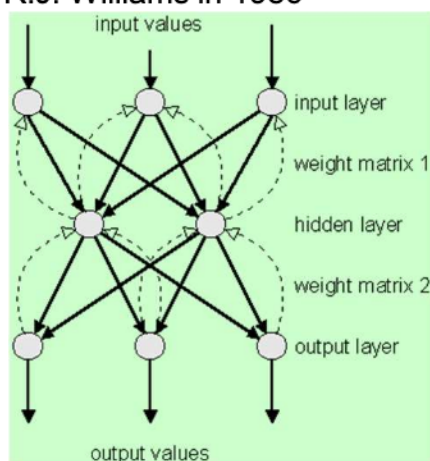
- 1 input layer
- 1 or more hidden layers
- 1 output layer

Input value types:

Binary

Learning Method:

BackPropagation
(Supervised)



[6088 - V323.indd](#) Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986).

Learning representations by back-propagating errors. *nature*, 323(6088), 533-536.

Learning representations by back-propagating errors

Original
paper

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California,
San Diego, La Jolla, California 92093, USA

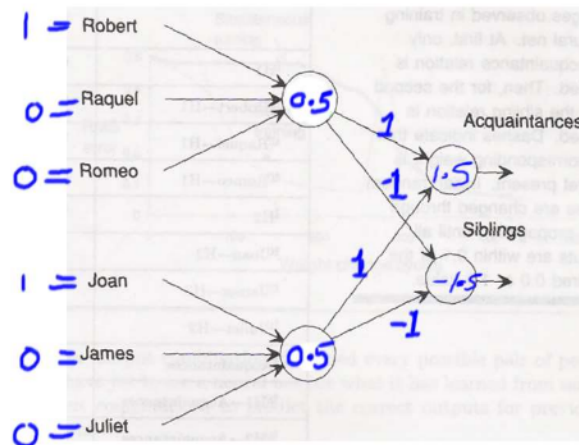
† Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, Philadelphia 15213, USA

[6088 - V323.indd](#)

Rumelhart, D. E.,
Hinton, G. E., &
Williams, R. J. (1986).
Learning
representations by
back-propagating
errors. *nature*, 323(60
88), 533-536.

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

Backpropagation Networks Example:



Weights from unit 1 and 2 in hidden layer to unit 1 in o/p layer= 1

Weights from unit 1 and 2 in hidden layer to unit 2 in o/p layer= -1

Threshold of unit 1 & 2 in the hidden layer=0.5

Threshold of unit 1 in the output layer= 1.5

Threshold of unit 2 in the output layer= -1.5

Source: AI by Patrick Henry Winston book; Page no: 448

Gradient Descent Rule
for Single Neuron or
1-layer FFNN
(setting=regression)

DELTA RULE

Gradient Descent

- Although the perceptron rule finds a successful weight vector when the training examples are linearly separable, **it fails to converge if the examples are not linearly separable.**
- Another training rule called **Delta Rule** was designed to overcome this difficulty.

Gradient Descent and the Delta Rule

- The key idea behind the delta rule is to use **gradient descent to search the hypothesis space of possible weight vectors to find the weights that best fit the training examples.**
- This rule is important as it provides the basis for Backpropagation algorithm.
- The delta training rule is best understood by considering the task of training an unthresholded perceptron, i.e. linear unit for which the output O is given by :

$$O(\vec{x}) = \vec{w} \cdot \vec{x} \cdots \cdots (1)$$

- A measure of the training error $E(\vec{w})$ is simply defined as half the squared difference between the target output t_d and the linear output O_d , summed over all training examples.

Delta Rule for Single Unit/Neuron

- A measure for the training error of a hypothesis or weight vector relative to the training examples is:

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \cdots \cdots (2)$$

where D is the set of training examples, t_d is the target output for training example d, and O_d is the output of the linear unit for training example d.

- Error E is characterized as a function of $\vec{w}$ because the linear unit output O depends on this weight vector.
- E also depends on the particular set of training examples, but we assume these are fixed during training, so we do not bother to write E as an explicit function of these.

Visualizing the Hypothesis Space:

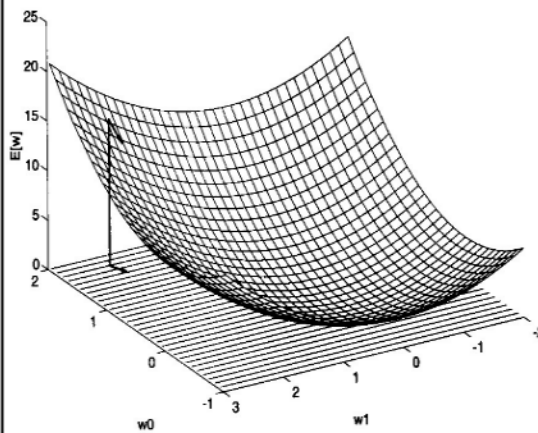


FIGURE 4.4

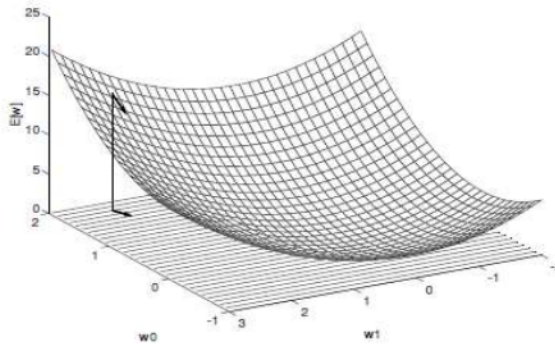
Error of different hypotheses. For a linear unit with two weights, the hypothesis space H is the w_0, w_1 plane. The vertical axis indicates the error of the corresponding weight vector hypothesis, relative to a fixed set of training examples. The arrow shows the negated gradient at one particular point, indicating the direction in the w_0, w_1 plane producing steepest descent along the error surface.

- To understand the gradient descent algorithm it is helpful to visualize the entire hypothesis space of possible weight vectors and their associated E values, as illustrated in figure.
- This error surface is parabolic with a single global minimum.

Contd.

- For a linear unit with two weights the hypothesis space H is the w_0, w_1 plane.
- The vertical axis indicates the error of the corresponding weight vector hypothesis, relative to a fixed set of training examples.
- The arrow shows the negated gradient at one particular point, indicating the direction in the w_0, w_1 plane producing steepest descent along the error surface.
- Gradient descent search determines a weight vector that minimizes Error E by starting with an arbitrary initial weight vector, then repeatedly modifying it in small steps.
- At each step, the weight vector is altered in the direction that produces the steepest descent along the error surface depicted in figure in previous slide.
- This process continues until the global minimum error is reached.

Gradient Descent



Gradient

$$\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n} \right]$$

Training rule:

$$\Delta \vec{w} = -\eta \nabla E[\vec{w}]$$

i.e.,

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

Gradient Descent

$$\begin{aligned} \frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_d (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\ &= \sum_d (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \vec{w} \cdot \vec{x}_d) \\ \frac{\partial E}{\partial w_i} &= \sum_d (t_d - o_d) (-x_{i,d}) \end{aligned}$$

GRADIENT-DESCENT(*training_examples*, η)

Each training example is a pair of the form $(\vec{x}, t)$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., .05).

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
 - Initialize each Δw_i to zero.
 - For each $(\vec{x}, t)$ in *training_examples*, Do
 - Input the instance $\vec{x}$ to the unit and compute the output o
 - For each linear unit weight w_i , Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i \quad (\text{T4.1})$$

- For each linear unit weight w_i , Do

$$w_i \leftarrow w_i + \Delta w_i \quad (\text{T4.2})$$

- Termination Condition:
 - To minimize the root mean squared error

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \dots\dots\dots (2)$$

Efficiency of Gradient Descent

- Because the error surface contains only a single global minimum, this algorithm will converge to a weight vector with minimum error, regardless of whether the training examples are linearly separable, given a sufficiently small learning rate is used.
- If learning rate is too large, gradient descent search runs the risk of overstepping the minimum in the error surface rather than settling into it.
 - For this reason, one common modification to the algorithm is to gradually reduce the value of learning rate as the no. of gradient descent steps grows.

Representational power of Perceptrons

- Single Perceptron can represent all of the primitive boolean functions that are linearly separable e.g.) AND, OR, NAND, NOR.
- **Unfortunately some functions can not be represented by a single neuron/perceptron or single layer network, such as XOR function, even though you apply powerful learning algorithm such as delta training rule.**

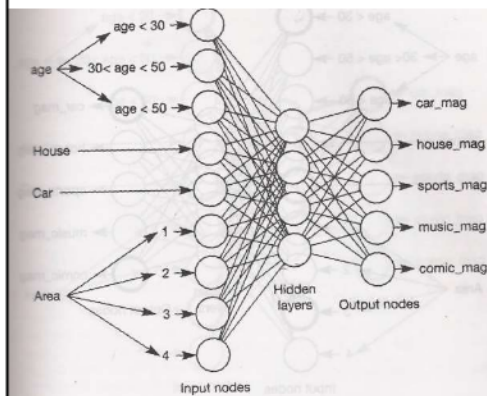
Error Backpropagation method MULTILAYER FEED-FORWARD NEURAL NETWORK (FFNN)

[6088 - V323.indd](#) Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986).
Learning representations by back-propagating errors. *nature*, 323(6088), 533-536.

Applications of Backpropagation or Multi-layer FFNN

- FFNN learning is well suited to the problems in which the data corresponds to noisy, complex sensor data, such as inputs from cameras and microphones.
- It is appropriate for problems with the following characteristics:
 - Instances are represented by many attribute value pairs.
 - Long training times are acceptable.
 - Target function output may be discrete valued, real valued, or a vector of several real or discrete valued attributes.

Applications of Backpropagation Network for Simple Data Mining:



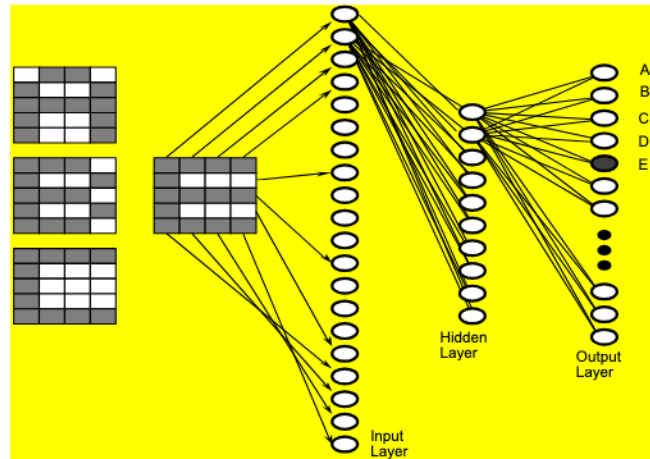
Source: Data Mining by Peter and Dolf

• ANN learning is well suited to the problems in which the data corresponds to noisy, complex sensor data, such as inputs from cameras and microphones.

• It is appropriate for problems with the following characteristics:

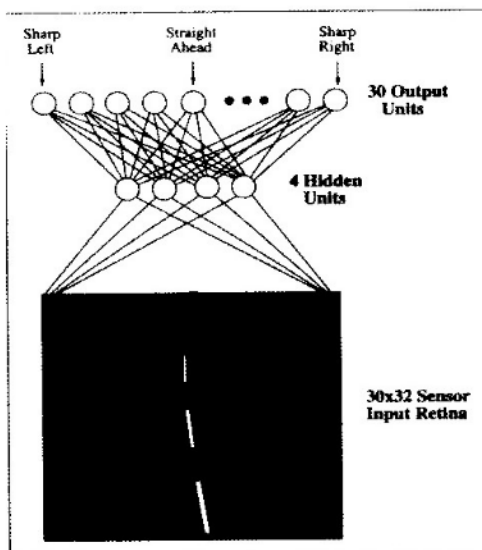
- Instances are represented by many attribute value pairs.
- Long training times are acceptable.
- the target function output may be discrete valued, real valued, or a vector of several real or discrete valued attributes.

Optical Character Recognition



The MNIST database of handwritten digits, available from this page, has a training set of 60,000 examples, and a test set of 10,000 examples
 MNIST dataset link: <http://yann.lecun.com/exdb/mnist/>

The ALVINN System by Dean A. Pomerleau Carnegie Mellon University

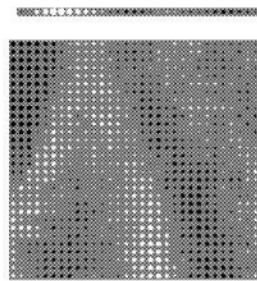
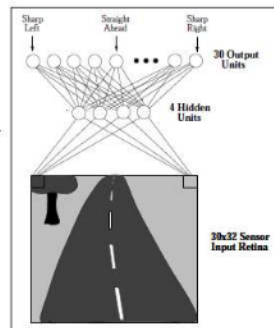


- It's a 3-layer backpropagation Neural Network learning to steer an autonomous vehicle.
- ALVINN system uses BP method to learn to steer an autonomous vehicle driving at speeds up to 70 miles per hour.
- The image of a forward mounted camera is mapped to 960 neural network inputs, which are fed forward to 4 hidden units, connected to 30 output units.
- Network outputs encode the commanded steering direction.

ALVINN drives at 70 m/h

Autonomous Land Vehicle in a Neural Network;

source: CMU



Samples from the driving dataset

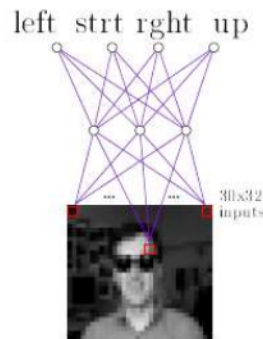


<http://research.comma.ai>

<https://www.youtube.com/watch?v=KTrgRYa2wbl>

Neural Nets for Face Recognition

- To classify camera images of faces of various people in various poses.
- Images of 20 different people were collected, including approx. 32 images per person, varying the person's expression (happy, sad, angry, neutral), target function is: direction in which they were looking (left, right, straight ahead, up)



Typical input images

Some more examples

- Brainwave-sensing headsets need better security.
- Hackers can steal passwords by analysing brainwave signals (EEG). Your brainwaves are new key to hacking.
 - Researchers from University of California & Alabama collected EEG data from 12 healthy subjects while using a keyboard to type random PINs and passwords.
 - They found that after participants entered 200 characters while wearing an EEG headset, their algorithm could guess four-digit PINs with a 43.4% success rate and six-character passwords with a 37.3% success rate.
- AI can look at food photos and guess recipe.
 - Researchers collected a database of over 1 million recipes annotated with information about ingredients. They used dataset to train a neural network to learn patterns between the food images and corresponding ingredients and recipes.

[Hackers can steal PINs, passwords from your brainwaves: study - Daily Excelsior](#)
[Recipe1M+: A Dataset for Learning Cross-Modal Embeddings for Cooking Recipes and Food Images - MIT](#)

Watch, Attend and Spell S/W System

- WAS s/w system used computer vision and machine learning methods to learn to lip read from a dataset made up of more than 5000 hours of TV footage, gathered from 6 different programs.
- Oxford university scientists claimed it can lip-read more accurately than people, an advancement that would help people with hearing loss.
- March 20, 2017

An artificial intelligence (AI) system called "Watch, Attend, and Spell"; developed by researchers at Oxford University in collaboration with Google's [DeepMind AI unit](#); is able to lip read up to 50% of silent speech, according to a [BBC article](#).

Source: <https://deepmind.com/research/publications/2019/large-scale-visual-speech-recognition>

Multilayer FFNN/Perceptron and the Error Backpropagation Algorithm

- Single Perceptrons can only express linear decision surfaces for linearly separable data.
- **In contrast, multilayer networks trained using the Backpropagation algorithm are capable of expressing a rich variety of nonlinear decision surfaces.**
- It is possible for the multilayer network to represent highly nonlinear decision surfaces that **are much more expressive than the linear decision surfaces of single units/perceptron as used by McCulloch and Pitts Model.**

Backpropagation Procedure

- Feed forward multilayer networks are so powerful that they are able to solve a variety of problems.
- Back-propagation procedure combines *Hill Climbing* and *Gradient Descent*.
- **It does Hill Climbing by Gradient Descent.**
- **Hill Climbing** tries each possible step to choose a step that does the most good at every step.
- **Gradient Descent** tries moving in the direction of most rapid performance improvement.

A Differential Threshold Unit

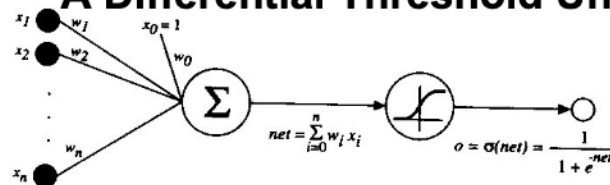
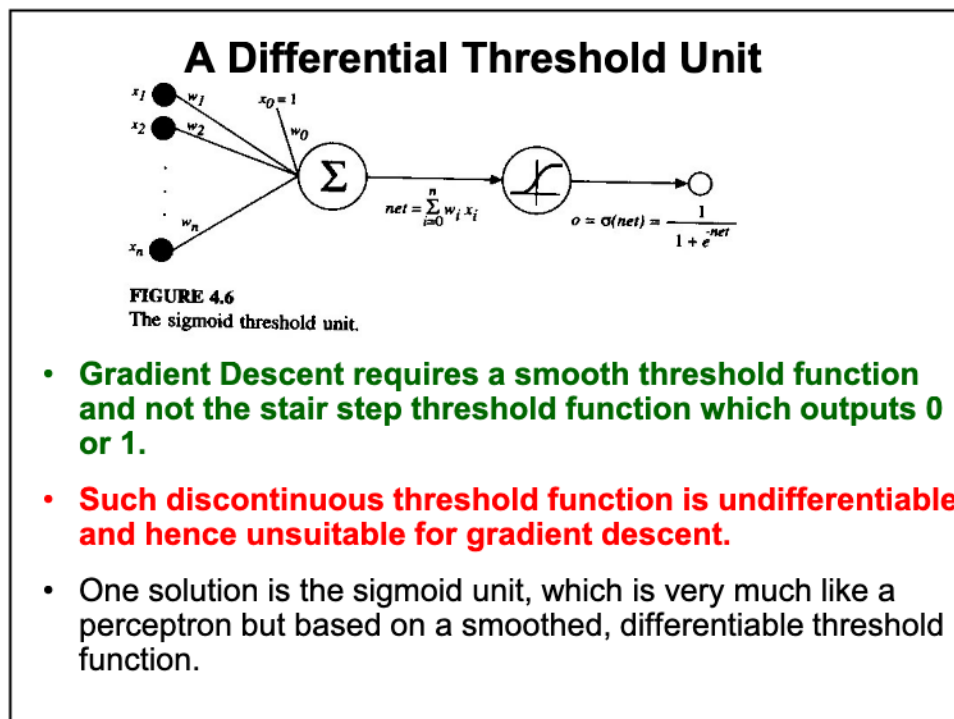
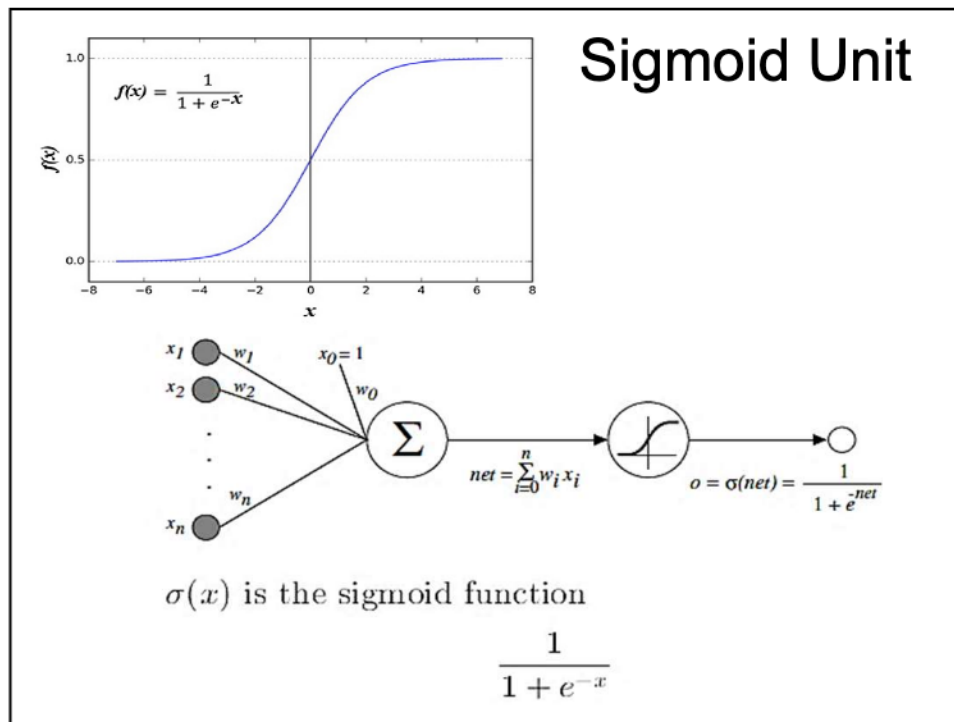
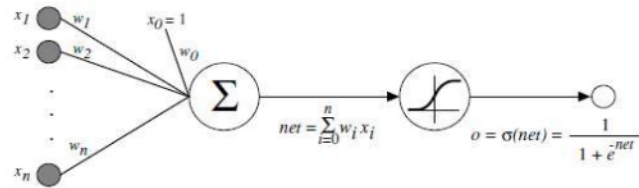


FIGURE 4.6
The sigmoid threshold unit.

- **Multiple layers of cascaded linear units still produce only linear functions.**
- **Gradient Descent requires a smooth error surface which can not be achieved with the stair step transfer function outputting 0 or 1.**
- For MLP we need units whose output is a non-linear function of its input, and also the output is a differential function of its inputs.
- One solution is the sigmoid unit, which is very much like a perceptron but based on a smoothed, differentiable threshold function.



Sigmoid Unit



$\sigma(x)$ is the sigmoid function

$$\frac{1}{1 + e^{-x}}$$

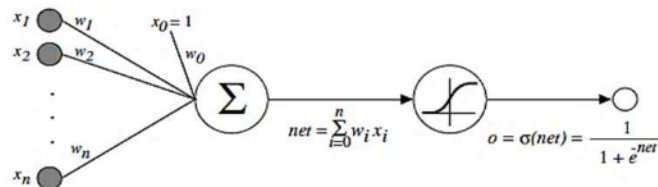
Nice property: $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$

We can derive gradient decent rules to train

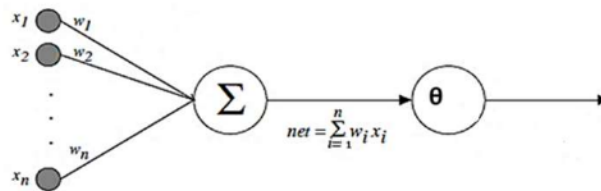
- One sigmoid unit
- *Multilayer networks* of sigmoid units → Backpropagation

Zero-threshold unit

- Backpropagation method assumes each unit to be a zero-threshold neuron.
- A non-zero threshold neuron is computationally equivalent to a zero-threshold neuron with an **extra link connected to an input known as bias** that is always held at -1 or 1.
- In this way **separate procedures will not be needed for adjusting weights and for adjusting thresholds.**



Non-zero threshold unit



For non-zero threshold unit:

$$\sum_{i=1}^n w_i x_i > \theta, \text{ Output} = 1 \text{ otherwise } 0$$

$$\sum_{i=1}^n w_i x_i - \theta > 0, \text{ Output} = 1 \text{ otherwise } 0$$

Bias as a special input tied to 1

- Bias is a measure of how easy/difficult it is to get the perceptron to fire.
 - Or to put it in more biological terms, the bias is a measure of how easy it is to get the perceptron to *fire*.
 - For a perceptron with a big bias, it's extremely easy for the perceptron to output a 1.
 - But if the bias is very negative, then it's difficult for the perceptron to output a 1.

Learning with Backpropagation

- Backpropagation method assumes each unit to be a zero threshold neuron.
- Non zero thresholds can be eliminated as a non zero threshold neuron is computationally equivalent to a zero threshold neuron with an **extra link connected to an input known as bias** that is always held at -1 or 1.
- In this way **separate procedures/training rules will not be needed for adjusting weights and for adjusting thresholds/biases.**

Backpropagation Method

- The Back propagation procedure computes:
 - changes to the weights in the final layer first,
 - reuses much of the same computation to compute changes to the weights in the ultimate layer and
 - ultimately goes back to the initial layer.
- A big change in w makes sense if that change can reduce a large output error and if the size of that reduction is substantial.

Error Backpropagation Rule derivation

Source: Tom M. Mitchell

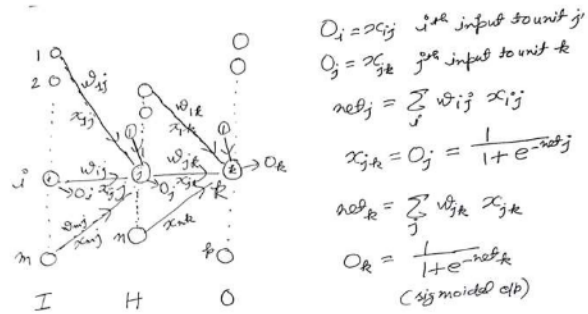
B.P. Algorithm

- It learns the weights for a multilayer network, given a network with a fixed set of units and interconnection.
- This algo considers networks with multiple output units rather than single units, so here E (error) is defined to sum the errors over all of the network output units:

$$E(w) = \frac{1}{2} \sum_{d \in D} \sum_{K \in \text{outputs}} (t_{kd} - o_{kd})^2$$

- where outputs is the set of output units in the network, t_{kd} and O_{kd} are the target and output values associated with the k_{th} output unit and training example d .
- The learning problem faced by B.P. is to search a large hypothesis space defined by all possible weight values for all the units in the network.

Backpropagation



Error at O/P layer with 'p' o/p units:

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} \sum_{k \in O/P} (t_k - O_k)^2$$

Error for each training example d :

$$E_d(\vec{w}) = \frac{1}{2} \sum_{k \in O/P} (t_k - O_k)^2$$

$$\Delta w_{ij} = -\eta \frac{\partial E_d}{\partial w_{ij}}, \text{ As } w_{ij} \text{ can influence } t_k \text{ through } net_j \text{ only through } net_j$$

$$= -\eta \frac{\partial E_d}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ij}}, \text{ As } net_j = \sum_i w_{ij} x_{ij}$$

$$\Delta w_{ij} = -\eta \frac{\partial E_d}{\partial net_j} \cdot x_{ij} \quad (1)$$

Training Rule for output units

Case 1: Training Rule for O/P unit weights:

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial O_j} \cdot \frac{\partial O_j}{\partial net_j}, \text{ As } net_j \text{ can influence network through } O_j, \text{ here } O_j = \frac{1}{1 + e^{-net_j}}$$

$$= \frac{\partial}{\partial O_j} \left[\frac{1}{2} \sum_k (t_k - O_k)^2 \right] \cdot \frac{\partial O_j}{\partial net_j}$$

$$= \frac{\partial}{\partial O_j} (t_j - O_j) \cdot \frac{\partial O_j}{\partial net_j} \cdot \frac{\partial O_j}{\partial net_j}$$

$$\frac{\partial O_j}{\partial net_j} = \frac{\partial}{\partial net_j} \left(\frac{1}{1 + e^{-net_j}} \right) = O_j (1 - O_j)$$

$$\frac{\partial E_d}{\partial net_j} = -O_j (t_j - O_j) (1 - O_j)$$

By putting $\frac{\partial E_d}{\partial net_j}$ in eq (1) $[\Delta w_{ij} = -\eta \frac{\partial E_d}{\partial net_j} \cdot x_{ij}]$

$$\Delta w_{ij} = \eta O_j (t_j - O_j) (1 - O_j) x_{ij}$$

$$\Delta w_{ij} = \eta \delta_j x_{ij}$$

$$\text{here } \delta_j = -\frac{\partial E_d}{\partial net_j} = O_j (t_j - O_j) (1 - O_j) \quad (2)$$

If we use 'k' for o/p layer neurons:

$$\delta_k = O_k (t_k - O_k) (1 - O_k)$$

Training Rule for Hidden units

Case 2: Training Rule for Hidden Unit Weights when 'j' is an internal/hidden unit in the network. The derivation of training rule for w_{jk} must take into account the set of all units immediately downstream of unit 'j' in the network i.e. all units whose direct inputs include the CIP of unit 'j'.

$$\begin{aligned}
 \frac{\partial E_d}{\partial net_j} &= \sum_{k \in \text{downstream}(j)} \frac{\partial E_d}{\partial net_k} \cdot \frac{\partial net_k}{\partial net_j} \quad (\text{Use eq 2) from case 1}) \\
 &= \sum_{k \in \text{downstream}(j)} -\delta_k \frac{\partial net_k}{\partial net_j} \quad \text{here } \frac{\partial net_k}{\partial net_j} = \sum_j w_{jk} x_{jk} \\
 &\quad \left(\frac{\partial net_k}{\partial net_j} \cdot \frac{\partial net_j}{\partial net_j} \right) \\
 &= \sum_{k \in \text{downstream}(j)} -\delta_k w_{jk} \cdot \frac{\partial net_j}{\partial net_j} \rightarrow o_j(1-o_j) \\
 \frac{\partial E_d}{\partial net_j} &= o_j(1-o_j) \sum_{k \in \text{downstream}(j)} -\delta_k w_{jk} \\
 &\quad (\text{Multiply with '1' both sides}) \\
 \delta_j &= -\frac{\partial E_d}{\partial net_j} = o_j(1-o_j) \sum_{k \in \text{downstream}(j)} \delta_k w_{jk} \\
 \text{Now substitute } -\frac{\partial E_d}{\partial net_j} \text{ in eq (1)} \quad [\Delta w_{ij} &= -\eta \frac{\partial E_d}{\partial net_j} \cdot x_{ij}] \\
 \Delta w_{ij} &= [o_j(1-o_j) \sum_{k \in \text{downstream}(j)} \delta_k w_{jk}] x_{ij} \cdot \eta \\
 \Delta w_{ij} &= \eta \delta_j x_{ij}
 \end{aligned}$$

Backpropagation Algorithm

BACKPROPAGATION(training_examples, η , n_{in} , n_{out} , n_{hidden})

Each training example is a pair of the form $(\vec{x}, \vec{t})$, where $\vec{x}$ is the vector of network input values, and $\vec{t}$ is the vector of target network output values.

η is the learning rate (e.g., .05). n_{in} is the number of network inputs, n_{hidden} the number of units in the hidden layer, and n_{out} the number of output units.

The input from unit i into unit j is denoted x_{ji} , and the weight from unit i to unit j is denoted w_{ji} .

- Create a feed-forward network with n_{in} inputs, n_{hidden} hidden units, and n_{out} output units.
- Initialize all network weights to small random numbers (e.g., between $-.05$ and $.05$).

Source: Machine Learning Book by Tom Mitchell, TMH
(book downloadable)

Backpropagation Algorithm

When the termination condition is met, Do

For each $(\vec{x}, \vec{t})$ in *training_examples*, Do

Propagate the input forward through the network:

1. Input the instance $\vec{x}$ to the network and compute the output o_u of every unit u in the network.

Propagate the errors backward through the network:

2. For each network output unit k , calculate its error term δ_k

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k) \quad (\text{T4.3})$$

3. For each hidden unit h , calculate its error term δ_h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in \text{outputs}} w_{kh} \delta_k \quad (\text{T4.4})$$

4. Update each network weight w_{ji}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

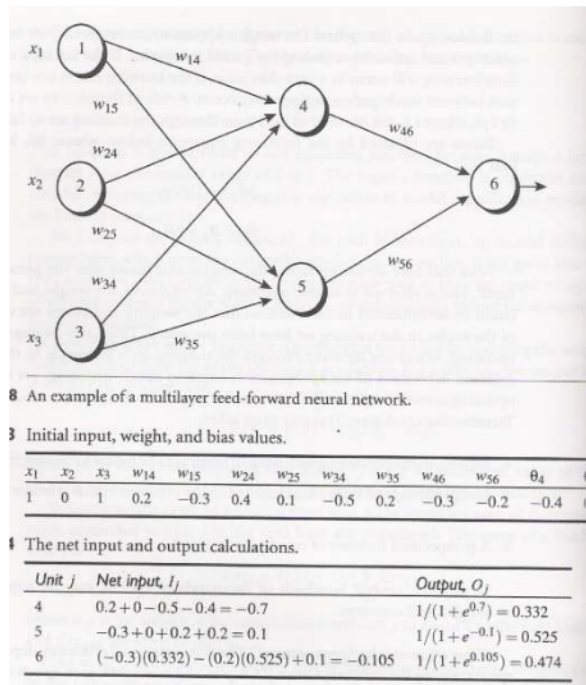
where

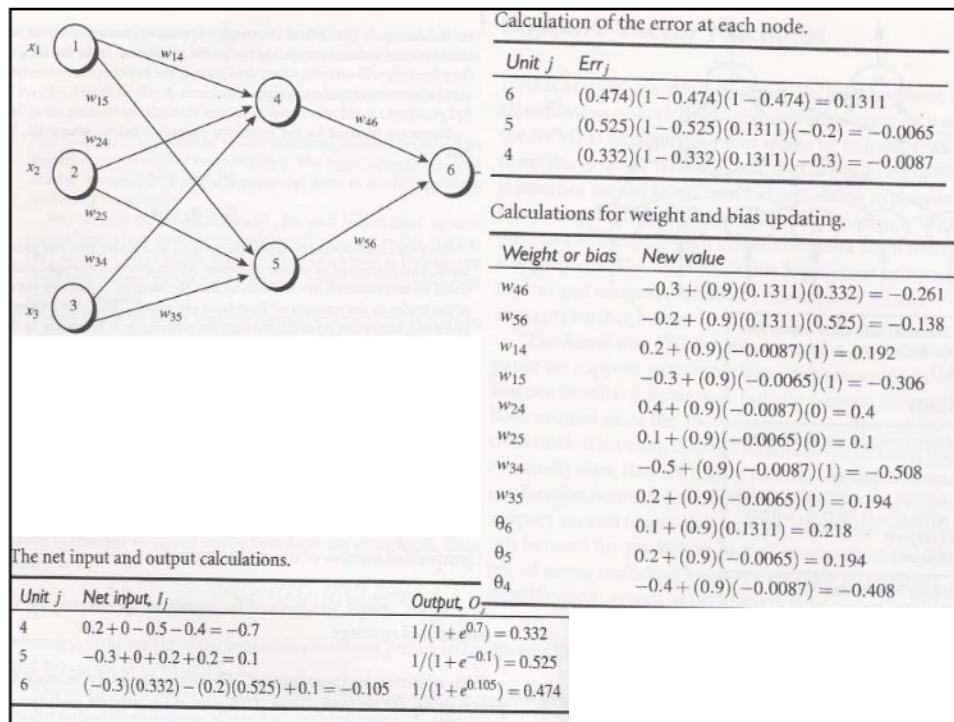
$$\Delta w_{ji} = \eta \delta_j x_{ji} \quad (\text{T4.5})$$

Example:
training example=
(1 0 1 $\rightarrow$ 1)
Weight updates
using Backpropagation
Method

Source: Han and Kamber

(Iteration 1)





One-layer (Single Neuron) v/s Multilayer Network

- One major difference in the case of **multilayer networks vs single neuron** is that the error surface can have multiple local minima, in contrast to the single minimum parabolic error surface shown in graph of a parabolic curve for a single unit in a previous slide.
- Unfortunately, this means that gradient descent is guaranteed only to converge toward some local minima, and not necessarily the global minimum error.
- **Despite the lack of assured convergence to the global minimum error, BACKPROPAGATION is a highly effective function approximation method in practice.**